

Funcionalidad 1: Autocompletado con Debounce

El problema a resolver es técnico: si lanzas una petición a la base de datos por cada pulsación de tecla, vas a generar un cuello de botella. El código soluciona esto interponiendo un retraso (debounce) y delegando la búsqueda a nivel de base de datos.

1. La intercepción en el Frontend (React) Todo inicia en `SearchAutocomplete.jsx`. Este componente tiene un input de texto atado al estado local `query` que se actualiza con cada pulsación mediante el evento `onChange`.

Sin embargo, ese estado crudo no se usa para buscar. Se pasa al custom hook `useDebounce`, alojado en `useDebounce.js`. Esta función recibe el valor tipeado y un retraso de 300 milisegundos. En su interior, utiliza `setTimeout` para actualizar un nuevo estado (`debouncedValue`) solo cuando el tiempo asignado haya terminado. La función clave aquí es el *cleanup* del hook (`return () => clearTimeout(handler);`): si el usuario presiona otra tecla antes de los 300ms, cancela el timer anterior y lo reinicia.

De vuelta en `SearchAutocomplete.jsx`, un `useEffect` monitorea específicamente la variable `debouncedQuery`. Cuando esta variable cambia (lo cual ocurre porque el usuario dejó de teclear por 300ms), recién entonces se ejecuta la función asíncrona `fetchClients()`, la cual realiza la petición HTTP GET apuntando a `/api/clients/search?q={valor}`.

2. La resolución en el Backend (Java/Spring Boot) La petición impacta en el `ClientController.java`, específicamente en el método `searchClients`, el cual recibe el parámetro de URL `q`.

Si la cadena de texto no está vacía, el controlador llama al repositorio: `clientRepository.findByNameContainingIgnoreCase(q)`. Esta función abstracta está definida en `ClientRepository.java`. Al extender de `JpaRepository` y usar la convención de nombres de Spring Data, esta función genera automáticamente y por debajo una consulta SQL con la cláusula `LIKE %q%` y convierte ambos lados (búsqueda y columna) a minúsculas para ignorar mayúsculas (por el `IgnoreCase`). Retorna la lista en JSON al frontend.

Funcionalidad 2: Envío Asíncrono de Correos

El envío de un correo mediante el protocolo SMTP es un proceso I/O bloqueante por definición. Puede tardar varios segundos dependiendo de la red y del servidor proveedor (Gmail en este caso).

1. Habilitación de concurrencia en el backend Para que la API no se quede colgada esperando al correo, el código habilita el soporte de ejecución asíncrona a nivel global. Esto ocurre en la clase base `PresentacionApplication.java` con la anotación

@EnableAsync. Esto le indica a Spring que prepare un gestor de hilos (Thread Pool) para tareas en segundo plano.

2. Petición y desacoplamiento (Controlador) El usuario completa el formulario en el componente ClientDetailCard.jsx y, al someterlo (onSubmit), se ejecuta handleSendNotification. Esta función dispara un fetch POST al endpoint /api/clients/{id}/notify pasándole el cuerpo del mensaje.

La petición llega al método notifyClient del ClientController.java. Este método valida que el cliente exista en la base de datos y luego llama al servicio: emailService.sendEmail(client.getEmail(), request.getSubject(), request.getBody()).

Aquí está la arquitectura clave: en el controlador, la ejecución no se detiene a esperar que el sendEmail finalice. Al ser un llamado asíncrono, el hilo HTTP pasa inmediatamente a la siguiente línea del controlador: return ResponseEntity.accepted().build();. Esto le devuelve al frontend el código de estado 202 (Accepted), indicando que la tarea fue recibida y encolada, liberando así al cliente al instante.

3. Ejecución real del envío (Servicio) Mientras el usuario ya recibió la respuesta de éxito en pantalla, el método sendEmail del EmailService.java comienza a correr en un hilo secundario gracias a la anotación @Async.

Dentro de la función:

- Se imprime en consola mediante Thread.currentThread().getName() el nombre del hilo actual, lo que demostrará en vivo que no es un hilo HTTP estándar (http-nio-8080-exec-*) sino un hilo de tareas (task-*).
- Se utiliza la función Thread.sleep(3000); para forzar un retraso arbitrario de 3 segundos simulando intencionadamente la latencia severa de la red o servidor.
- Terminado el retraso, instancia un objeto SimpleMailMessage, setea la información obtenida por parámetro e invoca javaMailSender.send(message), que es la función final del framework que despacha los bits por el puerto SMTP hacia Gmail.